

3. Planning and Decision Making (6 hrs)

3.1. STRIPS and PDDL

3.2. Hierarchical Planning

3.3. Probabilistic Planning

3.4. Multi-agent Planning

3.5. Utility Theory

3.5.1. Utility Function

3.5.2. Multi-attribute utility Function

3.6. Decision Networks

3.6.1. Sequential Decision Problems

3.6.2. Algorithms for MDPs

3.6.3. Partial Observable MDP

What is Planning in AI?

Planning in artificial intelligence is the process of **deciding a sequence of actions** that an agent should take to achieve a particular goal. It is part of **deliberative reasoning**, where an agent “thinks ahead” before acting.

Planning assumes:

- The world is **described symbolically**
- The effects of actions are **predictable**
- Time and change can be **simulated mentally**

Key Concepts in AI Planning

1. State:

- i. A complete description of the world at a specific time
- ii. Represented as a set of logical facts (e.g., On(A, B), Clear(C))

2. Action / Operator:

- i. Represents a change from one state to another
- ii. Has:
 - **Preconditions:** What must be true to apply the action
 - **Effects:** What changes after the action

Contd..

3. Initial State:

- The current known state from which planning begins

4. Goal State:

- One or more conditions that we want to make true

5. Plan:

- A sequence (or sometimes graph) of actions that leads from the initial to the goal state

6. State Space:

- The set of all possible states the agent can be in
- Each state is a node; each action is a transition (edge)

Formal Definition of a Planning Problem:

A classical planning problem can be defined as a 5-tuple:

$$\mathbf{P} = (\mathbf{S}, \mathbf{A}, \gamma, \mathbf{s}_0, \mathbf{G})$$

Where:

$\mathbf{S}$ = Set of all possible states

$\mathbf{A}$ = Set of actions

γ = Transition function $\gamma(s, a) \rightarrow s'$

$\mathbf{s}_0$ = Initial state

$\mathbf{G}$ = Goal condition(s)

The solution is a sequence of actions $\langle a_1, a_2, \dots, a_n \rangle$ such that:

$$\gamma(\dots\gamma(\gamma(s_0, a_1), a_2)\dots, a_n) \models \mathbf{G}$$

Real-Life Example: Planning to Make Tea

Term	AI Definition	Tea Example
State	World at a moment, described by predicates	WaterInKettle, StoveOff
Action	Causes state transitions	BoilWater()
Preconditions	Must be true before action	WaterInKettle, StoveOn
Effects	New facts added/removed by action	Add: WaterBoiled
Initial State	Starting point	WaterInKettle, StoveOff, TeaBagInCup
Goal State	What needs to be true in final state	TeaReady
Plan	Sequence of actions	TurnOnStove → BoilWater → PourBoiledWater
State Space	Graph of all states and actions	All possible kitchen configurations
Domain	General set of actions	All tea/coffee making actions
Problem	Specific initial/goal facts	Make one cup of tea in current setting

STRIPS

STRIPS: Stanford Research Institute Problem Solver

- Introduced in early 1970s for classical planning
- Used action-based formalism for planning agents

It models a world using:

- **Facts (predicates)** describing the state of the world
- **Actions (operators)** that transform one state into another

Each action in STRIPS is defined by:

- **Preconditions:** What must be true before the action can be executed
- **Effects:** What becomes true (add list) and what becomes false (delete list) after execution

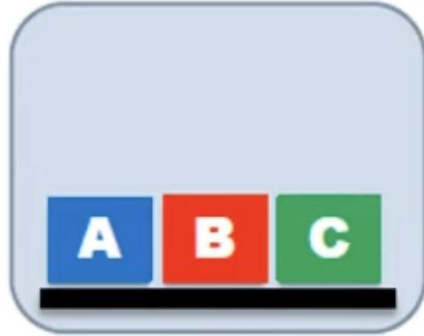
Example: Block World Domain (STRIPS)

Imagine a simple world with 3 blocks: A, B, and C.

We define:

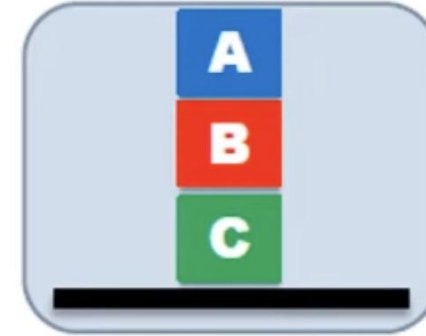
- **Initial State:**
 - On(A, Table)
 - On(B, Table)
 - Clear(A), Clear(B), Clear(C)
- **Goal State:**
 - On(A, B)
 - On(B, C)

STRIPS planning



On(A,Table)
On(B,Table)
On(C,Table)
Clear(A)
Clear(B)
Clear(C)

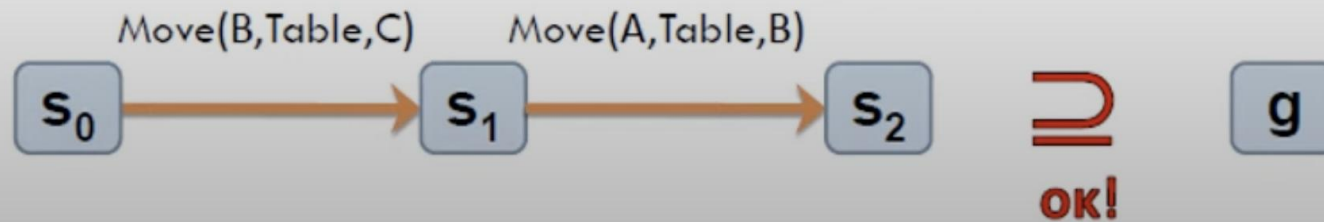
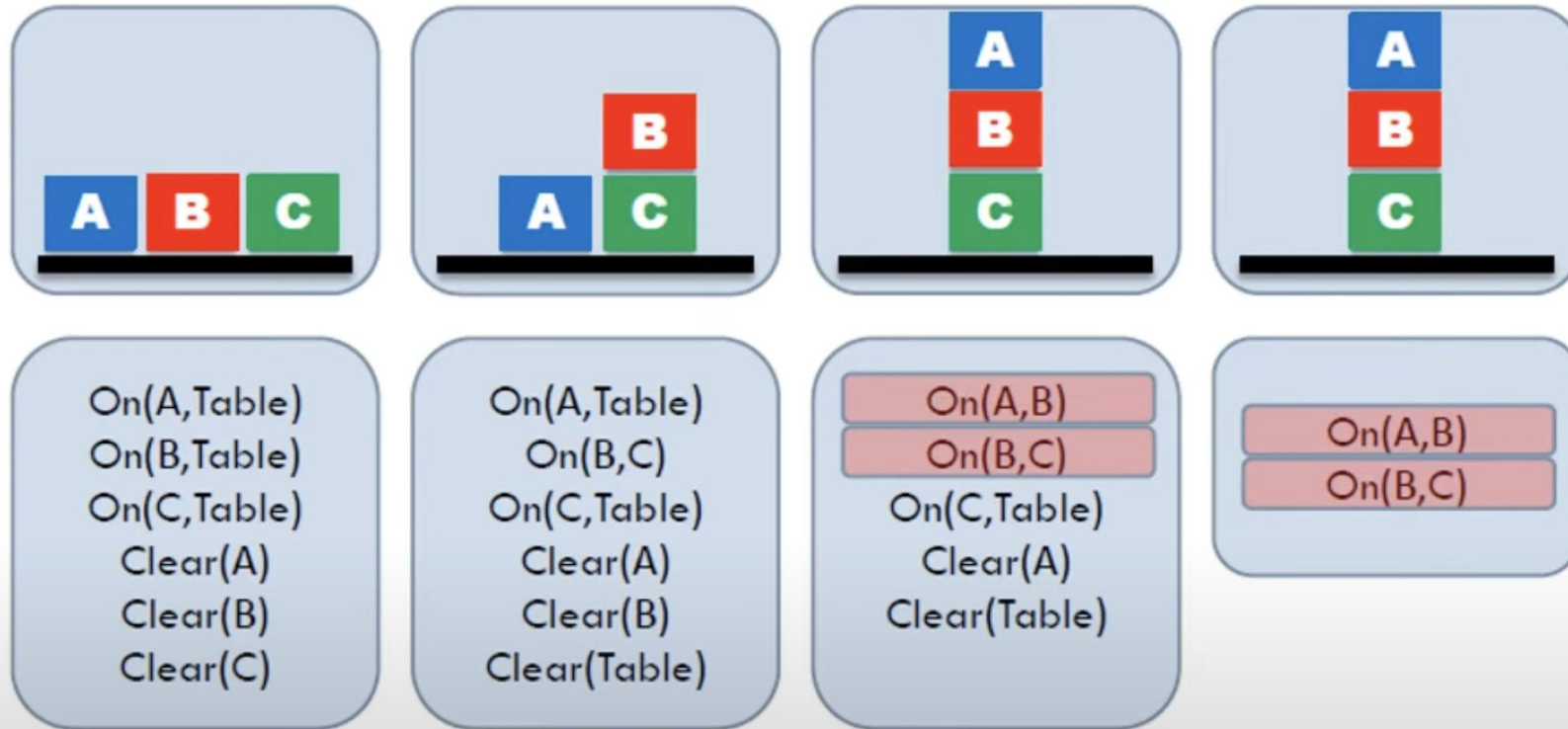
s_0



On(A,B)
On(B,C)

g

STRIPS planning



Summary: STRIPS

Concept	Explanation
Initial State	Blocks on table, all clear
Goal State	A on B, B on C
Actions Used	Move(B, Table, C), Move(A, Table, B)
Plan	Sequence of 2 move actions

STRIPS Planning Example: Air Cargo Transport

Scenario Overview

We want to move **cargos** between **airports** using **planes**. A cargo can be:

- At an airport
- In a plane

A plane can:

- Load cargo
- Fly between airports
- Unload cargo



What does this example mean?

It means:

- **We want an AI planning system to automatically generate a sequence of actions that can transfer cargo from one airport to another, using available planes and resources.**
- **So yes, it is about automating the transportation process not the physical movement, but rather the automated reasoning or planning about how to move cargo from A1 to A2.**

What's happening in the example?

Let's break it down in plain words:

1. **The world has facts:** where cargo is, where planes are.
2. **The AI planner knows what actions are allowed,** like:
 - i. Loading cargo into a plane
 - ii. Flying a plane
 - iii. Unloading cargo
3. **The goal is given:** “Cargo1 should be at A2 and Cargo2 at A1”.
4. **The AI planner thinks through the steps** needed to reach that goal:
 - i. “If I want Cargo1 at A2, and it's at A1, I should load it into Plane1 at A1, fly to A2, then unload it.”
 - ii. And so on.
5. **The planner outputs a plan a list of actions** to reach the goal.

1. Define Predicates

These are the facts that describe the **current state** of the world:

Prolog:

- $\text{At}(c, l)$; Cargo c is at location l
- $\text{In}(c, p)$; Cargo c is in plane p
- $\text{At}(p, l)$; Plane p is at location l

Variables:

- $c \rightarrow \text{cargo}$
- $p \rightarrow \text{plane}$
- $l \rightarrow \text{location (airport)}$

2. Initial State Example

Imagine:

- Cargo1 is at Airport A1
- Cargo2 is at Airport A2
- Plane1 is at Airport A1

Prolog:

At(Cargo1, A1)

At(Cargo2, A2)

At(Plane1, A1)

3. Goal State

We want:

- Cargo1 at A2
- Cargo2 at A1

Prolog:

At(Cargo1, A2)

At(Cargo2, A1)



4. STRIPS Actions

We'll define 3 main actions:

Action 1: Load(c, p, l)

(Load cargo c into plane p at location l)

Preconditions:

Prolog

- At(c, l)
- At(p, l)

Effects:

Add:

Prolog

- In(c, p)

Delete:

Prolog

- At(c, l)

Action 2: Fly(p, from, to)

(Fly plane p from airport from to airport to)

Preconditions:

Prolog

- At(p, from)

Effects:

Add:

Prolog

- At(p, to)

Delete:

prolog

- At(p, from)

Action 3: Unload(c, p, l)

(Unload cargo c from plane p at location l)

Preconditions:

Prolog

- In(c, p)
- At(p, l)

Effects:

Add:

Prolog

- At(c, l)

Delete:

Prolog

- In(c, p)

5. STRIPS Plan (Step-by-step)

Initial:

- At(Cargo1, A1)
- At(Cargo2, A2)
- At(Plane1, A1)

Goal:

- At(Cargo1, A2)
- At(Cargo2, A1)

Website To Run Prolog; <https://swish.swi-prolog.org/>

PDDL

PDDL (Planning Domain Definition Language) is a standard language developed in the late 1990s to provide a common framework for expressing planning problems. It is heavily inspired by STRIPS but much more expressive and modular.

PDDL = STRIPS + Extensions

It is composed of two files:

- 1.Domain File:** Defines the **predicates** and **actions** (the rules of the world)
- 2.Problem File:** Defines the **objects**, **initial state**, and **goal state**

Planning Domain Definition Language (PDDL)

- It is a **standard language** used to describe **AI planning problems**.
- Developed for the **International Planning Competitions (IPC)** to provide a unified way of defining planning tasks.
- Inspired by **STRIPS**, but more expressive and modular.

Why use PDDL?

- To **standardize** how planning problems are written.
- To allow **planning algorithms** to interpret and solve them.
- Makes it possible to separate:
 - **Domain knowledge** (general rules and actions)
 - **Problem instance** (specific goals and initial state)

Structure of a PDDL Problem

PDDL consists of two major parts:

1. Domain File (domain.pddl)

- Describes the **general rules** of the environment:
- Types of objects
- Predicates (facts)
- Actions (with parameters, preconditions, effects)

2. Problem File (problem.pddl)

- Describes the **specific planning instance**:
- Objects in the environment
- Initial state
- Goal state

DOMAIN FILE STRUCTURE

```
(define (domain air-cargo)
  (:requirements :strips :typing)

  (:types
    cargo airplane airport
  )

  (:predicates
    (at ?obj - (either cargo airplane) ?loc - airport)
    (in ?c - cargo ?p - airplane)
  )

  (:action load
    :parameters (?c - cargo ?p - airplane ?a - airport)
    :precondition (and (at ?c ?a) (at ?p ?a))
    :effect (and (not (at ?c ?a)) (in ?c ?p))
  )
```



```
  (:action unload
    :parameters (?c - cargo ?p - airplane ?a - airport)
    :precondition (and (in ?c ?p) (at ?p ?a))
    :effect (and (at ?c ?a) (not (in ?c ?p)))
  )

  (:action fly
    :parameters (?p - airplane ?from - airport ?to - airport)
    :precondition (and (at ?p ?from))
    :effect (and (not (at ?p ?from)) (at ?p ?to))
  )
)
```

PROBLEM FILE STRUCTURE

```
(define (problem cargo-problem)
  (:domain air-cargo)

  (:objects
    c1 c2 - cargo
    p1 - airplane
    a1 a2 - airport
  )

  (:init
    (at c1 a1)
    (at c2 a2)
    (at p1 a1)
  )

  (:goal
    (and (at c1 a2) (at c2 a1))
  )
)
```

How PDDL Works

Component	Purpose
(:types ...)	Define the object categories like cargo, airport, etc.
(:predicates ...)	Boolean functions describing state, e.g., at, in.
(:action ...)	Action template with parameters, preconditions, and effects.
(:init ...)	Initial world state.
(:goal ...)	Desired final state.

Real-world Use of PDDL

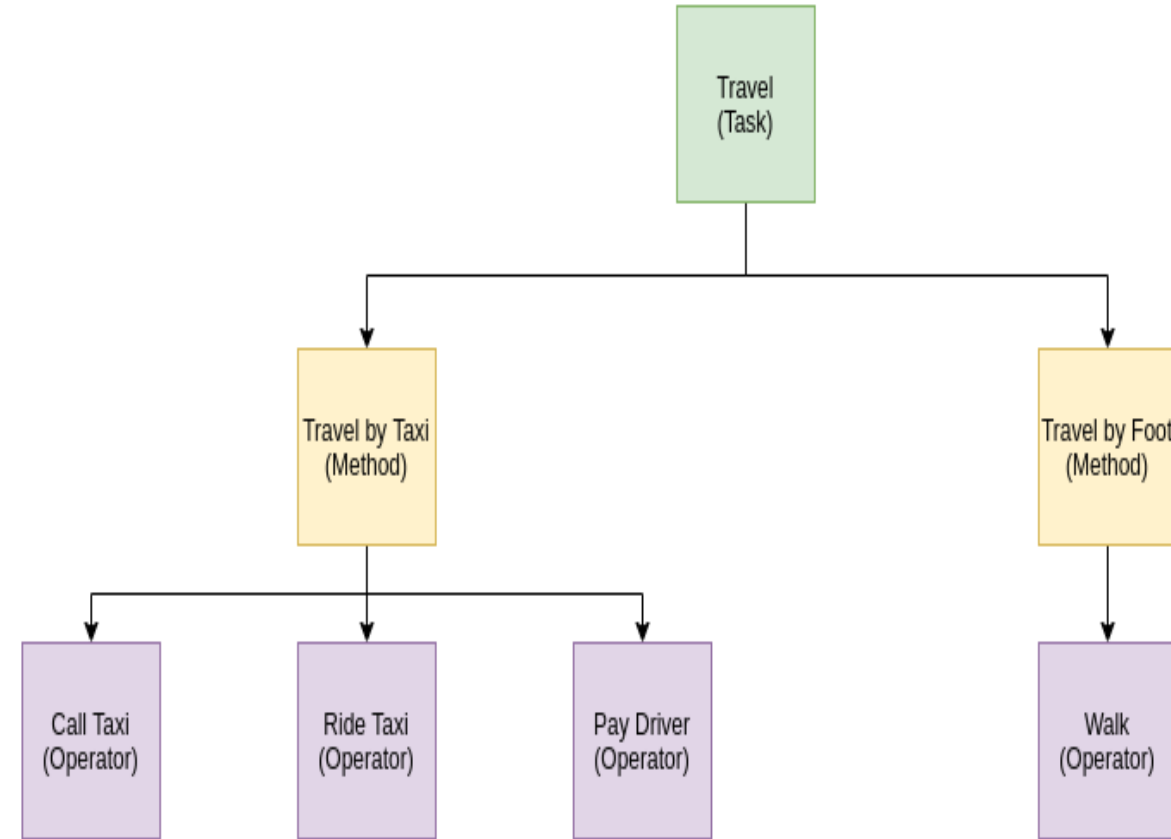
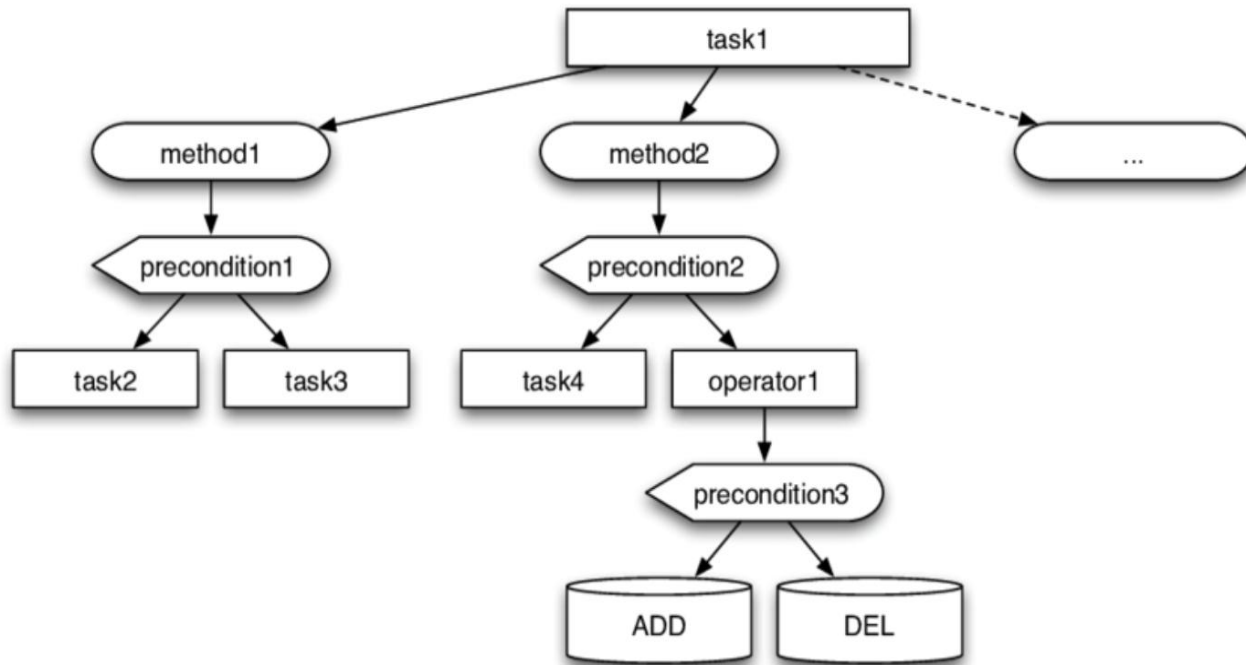
- NASA Mars Rover planning
- Autonomous warehouse robots
- Disaster response logistics
- Game AI planning agents

HIERARCHICAL PLANNING (HTN Planning)

- HTN Planning is based on the idea that **complex activities can be broken into smaller, manageable steps**. Just like a human might approach a task:
- To “prepare for a journey,” we plan:
 - Book tickets
 - Pack bags
 - Go to airport
- Each of those can be broken down further into atomic actions.

**HTN: Hierarchical Task Network.*

Figure: HTN Planning



Components of HTN

Component	Description
Task	High-level activity (e.g., deliver_package)
Primitive Task	Executable action (e.g., load, fly)
Compound Task	Non-executable high-level task (needs decomposition)
Method	Decomposition rule for compound tasks
Plan	A sequence of primitive tasks that satisfies the goal

HTN Planning vs Classical Planning

HTN Planning	Classical Planning
Top-down (task decomposition)	Bottom-up (goal to actions)
Task-based	State-based
Human-readable methods	Formal logic-based
More flexible, domain knowledge-rich	More general-purpose

Real-World Example: Package Delivery System

Goal: Deliver a package from Location A to Location B.

Compound Task:

- deliver_package (P, A, B)

Method to decompose:

Method: deliver_package P, A, B)

1. Pick_up (P, A)
2. Transport (P, A, B)
3. Drop_off (P, B)

Primitive Actions:

- Pick_up (P, A)
- Load (P, truck, A)
- Drive (truck, A, B)
- Unload (P, truck, B)
- Drop_off (P, B)

These actions are what the planner eventually returns as a **sequence of executable steps**.

Probabilistic Planning

- A planning approach used **when actions have uncertain outcomes**.
- Each action can lead to **multiple possible results**, each with a **probability**.
- The goal is to find a **policy** (not just a fixed plan) that maximizes the **expected utility** or chance of reaching the goal.

Key Characteristics

- **States**: Configurations of the world.
- **Actions**: May have **probabilistic effects** (not deterministic).
- **Transition Model**: Describes $P(s'|s, a)$ – The probability that action ‘a’ in state ‘s’ leads to state s'.
- **Goals**: Typically represented by a **utility function** or **desired goal states**.

Use Cases

- **Robotics** (noisy sensors and actuators)
- **Autonomous Vehicles** (uncertain traffic conditions)
- **Medical Diagnosis and Treatment Planning**
- **Disaster Response Robots**

Real-world Analogy: Robot Vacuum Cleaner

Imagine a robot vacuum cleaner that:

- Has to clean 3 rooms.
- Has a 90% chance of successfully cleaning a room when commanded.
- Sometimes the battery runs low and it skips the room (10%).

States:

- Clean(Room1), Dirty(Room2), etc.

Actions:

- Clean(Room1):
 - 90% → Room1 becomes clean
 - 10% → Room1 remains dirty

Transition Model:

- $P(\text{Room1}=\text{Clean} \mid \text{Room1}=\text{Dirty}, \text{action}=\text{Clean}(\text{Room1})) = 0.9$
- $P(\text{Room1}=\text{Dirty} \mid \text{Room1}=\text{Dirty}, \text{action}=\text{Clean}(\text{Room1})) = 0.1$

Goal:

- All rooms should be clean.

Policy:

- A mapping from state $\rightarrow$ action, e.g.,
"If Room1 is dirty, try to clean it. If clean, go to Room2."

Class Discussion:

Imagine you're planning to cross a river using a rope bridge in rain. You have 70% chance of success and 30% chance of falling. How would a probabilistic planner approach this decision?

Scenario Recap: Crossing a Rope Bridge in the Rain

- **Action:** Cross the rope bridge
- **Outcome 1 (Success):** 70% chance
- **Outcome 2 (Fall):** 30% chance
- **Goal:** Reach the other side safely
- **Risk:** Falling off the bridge (failure)

How a Probabilistic Planner Thinks

1. Model the Environment as a Probabilistic Space

A **probabilistic planner** doesn't assume the action will always succeed. It models **multiple outcomes with their probabilities**.

► States:

- Start: On this side of the river
- Crossed: Safely on the other side
- Fallen: Failed and fell into the river

► Action:

- AttemptCross(): Try to cross the bridge

Transition Probabilities:

Current State	Action	Next State	Probability
Start	AttemptCross()	Crossed	0.7
Start	AttemptCross()	Fallen	0.3

2. Attach Rewards or Utilities

In decision-making, planners evaluate not just outcomes, but **rewards** (or costs):

State	Utility (U) or Reward
Crossed	+100 (goal achieved)
Fallen	−200 (danger/injury)
Start	0

3. Calculate Expected Utility

- Probabilistic planners compute the **expected utility (EU)** of taking an action.
- $\text{EU}(\text{AttemptCross}) = 0.7 \text{ times } (+100) + 0.3 \text{ times } (-200) = 70 - 60 = +10$

The **expected utility is +10**, so a rational planner **may decide to cross**, unless a better/safer alternative exists.

4. Make Decision Based on EU or Policy

- If **EU > 0**, action is beneficial on average.
- If there are other safer options (e.g., "Wait", "Look for boat", "Build raft"), the planner compares their **expected utilities** too and chooses the best.

Example Extension: Add Another Option

New Option: Wait for Rain to Stop (adds delay but safer)

Action	Next State	Probability	Reward/Utility
WaitForClearWeather	Crossed	1.0	+100 – delay (say -20) = +80

Now compare:

- EU (Cross Now) = +10
- EU (Wait and Cross Later) = +80

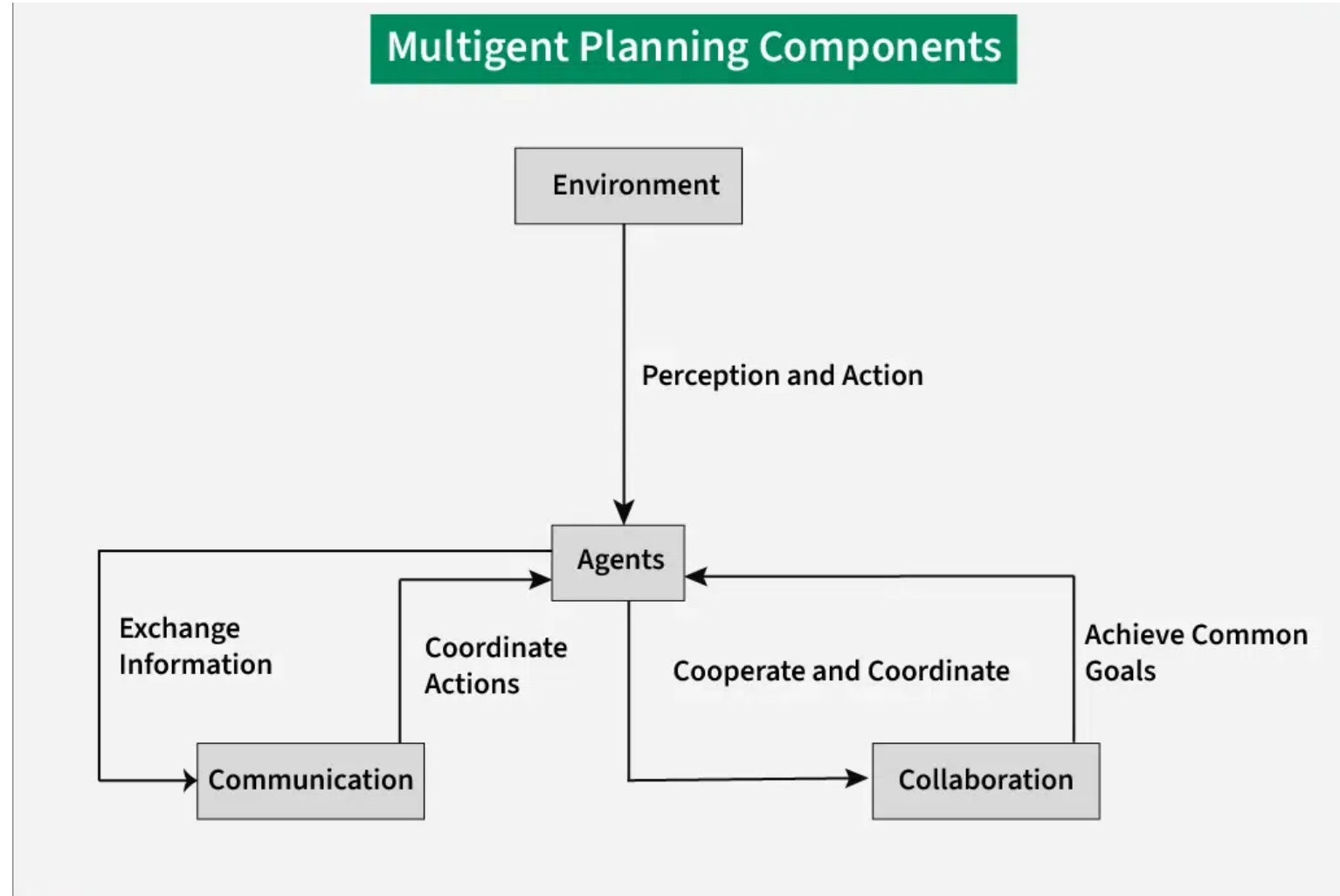
Planner chooses to wait.

Multi-agent Planning

Introduction

Multi-agent planning is the process of **generating, coordinating, and executing plans for multiple autonomous agents** operating in a shared environment. Unlike single-agent planning, the actions and decisions of one agent can **directly influence** the states and options available to others.

.



Contd..

Agents in this setting can be:

- **Cooperative** – Working together toward a shared goal.
- **Competitive** – Pursuing conflicting goals.
- **Mixed-motive** – Some cooperation, some competition.

Multi-agent planning is a core area in **distributed AI, robotics, and game theory**, and it is increasingly important for real-world applications like self-driving cars, autonomous drones, distributed sensor networks, and multi-robot manufacturing

Why Multi-agent Planning is Different from Single-agent Planning

In single-agent planning:

- The environment is passive only changes when the agent acts.
- The planner only needs to model its own capabilities and environment dynamics.

In multi-agent planning:

- **Dynamic environment:** Other agents act concurrently, changing the state unpredictably.
- **Interdependence:** The success of one agent's actions may depend on another's actions.
- **Partial control:** No single agent controls the entire system.

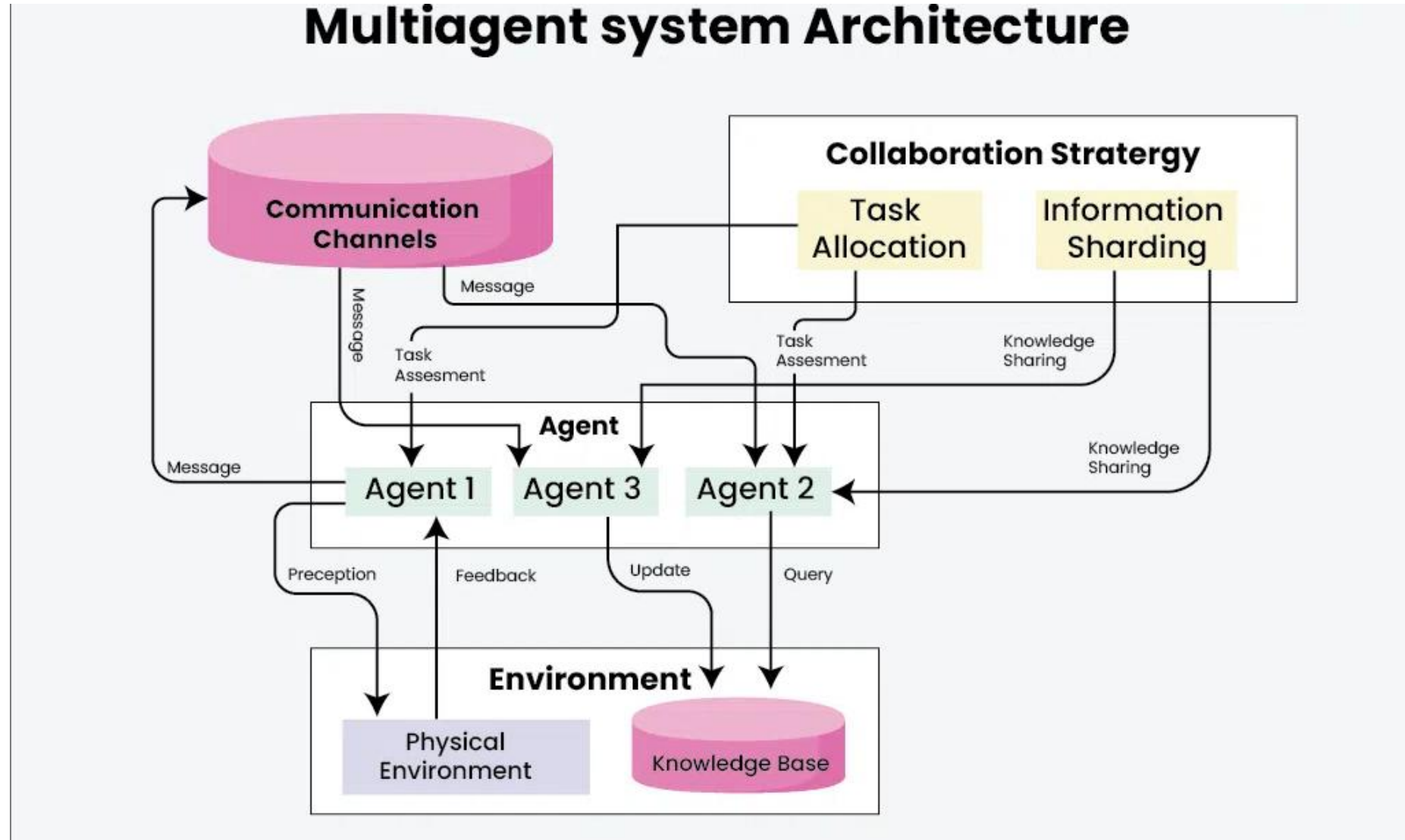
Formal Problem Definition

- A multi-agent planning problem can be formally defined as a tuple:
- $P = \langle N, S, A, T, G, C, O \rangle$

Where:

- N : Set of agents.
- S : Set of possible world states.
- A : Set of joint actions (a combination of each agent's action).
- $T(s, a)$ State transition function defines $P(s'|s, a)$
- G : Goals may be shared or individual.
- C : Constraints physical, logical, or resource limitations.
- O : Observation model what each agent can perceive about the state.

Figure: Multiagent System Architecture



Planning Architectures

Centralized Planning

- One planner has full knowledge of all agents' goals, abilities, and states.
- Generates a **joint plan** for all agents.
- Pros: Optimized, globally consistent.
- Cons: Scalability issues, single point of failure, requires full knowledge.

Decentralized Planning

- Each agent plans individually using local knowledge.
- Agents coordinate through **communication protocols** or **negotiation**.
- Pros: Scalable, robust to failures.
- Cons: Risk of conflicts or inefficiency.

Types of Multi-agent Planning

1. Cooperative Planning

- Agents share a **common goal** and coordinate to maximize collective utility.
- Example: Multiple rescue robots searching different areas for survivors.
- Planning is often **centralized** or **distributed but coordinated**.

2. Competitive Planning

- Agents have **opposing goals**.
- Requires **adversarial reasoning**: anticipating and countering opponents' strategies.
- Example: Chess, military simulations, autonomous trading bots.
- Uses **game theory** concepts: Minimax, etc.

3. Mixed-motive Planning

- Goals overlap partially agents cooperate in some parts, compete in others.
- Example: Logistics companies sharing warehouse space but competing for customers.

Formal Models for Multi-agent Planning

1. Multi-agent MDP (MMDP)

- Extension of MDP for multiple agents.
- Assumes **full observability** and **shared goals**.

2. Decentralized POMDP (Dec-POMDP)

- Partial observability.
- Each agent receives local observations.
- Joint decision-making is required to maximize shared utility.

3. Game-theoretic Planning

- Models strategic interactions.
- Useful in competitive or mixed settings.

4. Joint Intention Theory

- Formalizes how agents form, maintain, and drop shared commitments.

Algorithms & Approaches

- **Distributed Constraint Satisfaction (DisCSP):** Each agent solves part of the problem, coordinating to satisfy constraints.
- **Plan Merging:** Each agent makes a plan individually; plans are merged with conflict resolution.
- **Market-Based Planning:** Agents "bid" for tasks or resources.
- **Negotiation-Based Planning:** Agents propose and modify plans through negotiation.
- **Learning-Based Planning:** Agents use Reinforcement Learning to adapt strategies over time.

HTN-Style Multi-agent Planning Example

Scenario: Multi-drone disaster relief supply delivery

Problem Description

A major flood has isolated three villages (**V1**, **V2**, **V3**) from road access. Two autonomous drones (**D1** and **D2**) must deliver relief packages.

Constraints:

- Each drone can carry **only one package at a time**.
- Batteries need to be recharged after two trips.
- Villages have **priority levels** (V1 highest).
- Weather conditions affect flight times but are predictable.

1. High-level Goal (Root Task)

Deliver all relief packages to V1, V2, and V3.

2. Decomposition into Subtasks (HTN hierarchy)

Level 1 – Strategic Tasks

a. Plan Delivery Order

- Prioritize V1 → V2 → V3.

b. Assign Villages to Drones

- Minimize travel time and balance workload.

Contd..

Level 2 – Operational Tasks

For each delivery:

- **PickUpPackage(Location = Warehouse)**
- **FlyToVillage (Vi)**
- **DropPackage (Vi)**
- **ReturnToWarehouse**
- **RechargeIfNeeded(Drone)**

Contd..

Level 3 – Primitive Actions

- (Directly executable by drones)
- **TakeOff(Drone)**
- **NavigatePath(Start, Destination)**
- **Land(Drone)**
- **LoadPackage(PackageID)**
- **UnloadPackage(PackageID)**
- **RechargeBattery(Drone)**

3. State Representation

- Initial State:

At(D1, Warehouse)

At(D2, Warehouse)

BatteryFull(D1), BatteryFull(D2)

PackageAt(P1, Warehouse) → For V1

PackageAt(P2, Warehouse) → For V2

PackageAt(P3, Warehouse) → For V3

Weather(Clear)

Goal State:

PackageAt (P1, V1)

PackageAt (P2, V2)

PackageAt (P3, V3)

4. Planning with Coordination

Step 1 – Task Allocation (Joint Planning Level)

- Assign **D1** → **V1 & V3** (closest route loop)
- Assign **D2** → **V2** (mid-range trip, no interference with D1 path)

Step 2 – Parallel Plan Execution

Time Step	D1 Action	D2 Action
T1	PickUpPackage(P1)	PickUpPackage(P2)
T2	FlyToVillage(V1)	FlyToVillage(V2)
T3	DropPackage(P1)	DropPackage(P2)
T4	ReturnToWarehouse	ReturnToWarehouse
T5	PickUpPackage(P3)	Idle (Battery recharge)
T6	FlyToVillage(V3)	Idle
T7	DropPackage(P3)	Idle

5. Coordination Points

- **Conflict Avoidance:**

- Drones avoid mid-air collision by pre-planning separate flight corridors.

- **Battery Management:**

- Recharge schedule built into plan so that D2 recharges during D1's last trip.

- **Resource Sharing:**

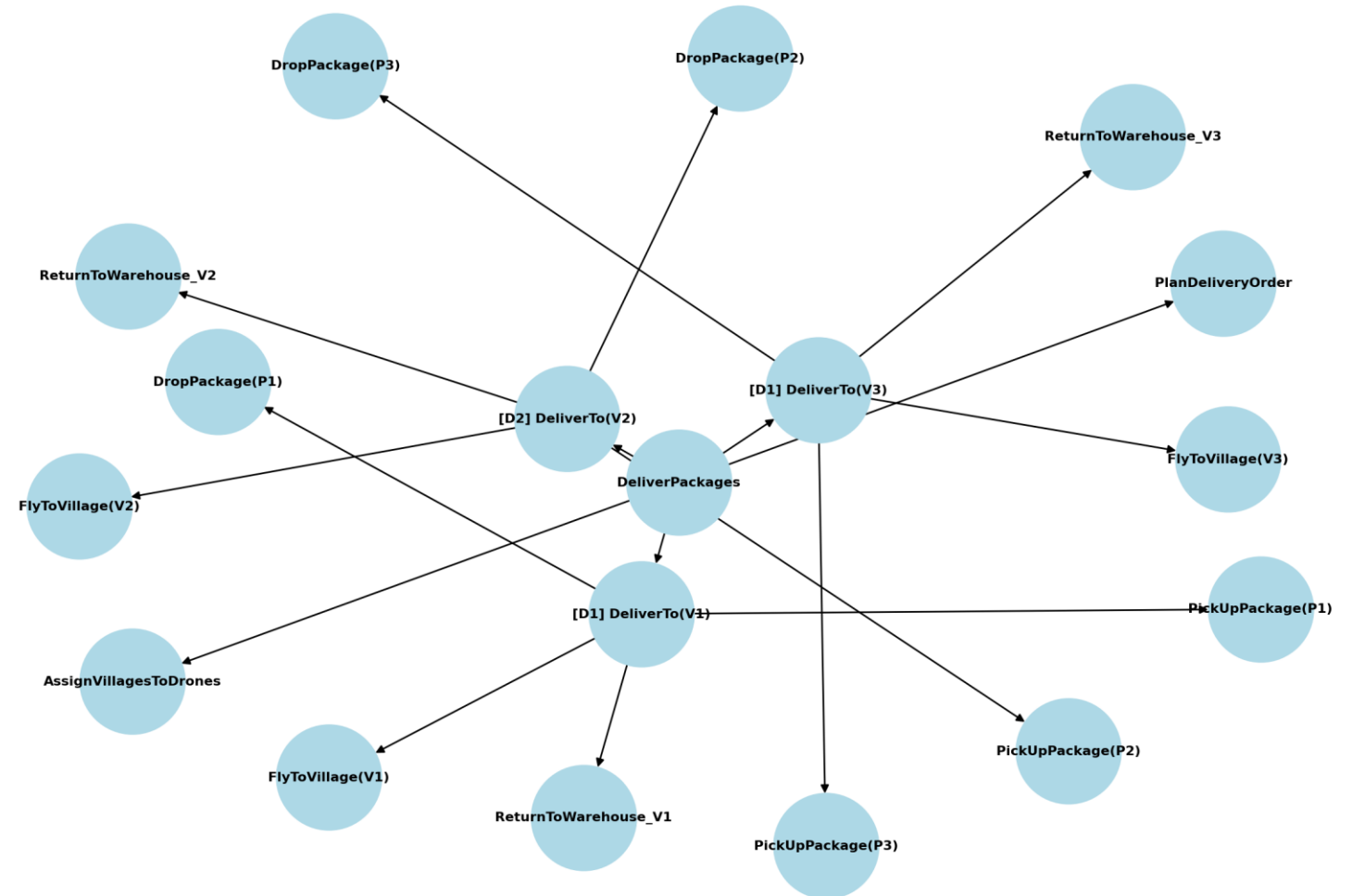
- Only one loading dock at warehouse → stagger package pickup times.

6. Final Joint Plan (HTN Tree)

DeliverPackages

- ├ PlanDeliveryOrder
- ├ AssignVillagesToDrones
- ├ [D1] DeliverTo(V1)
 - ├ PickUpPackage(P1)
 - ├ FlyToVillage(V1)
 - ├ DropPackage(P1)
 - └ ReturnToWarehouse
- ├ [D2] DeliverTo(V2)
 - ├ PickUpPackage(P2)
 - ├ FlyToVillage(V2)
 - ├ DropPackage(P2)
 - └ ReturnToWarehouse
- ├ [D1] DeliverTo(V3)
 - ├ PickUpPackage(P3)
 - ├ FlyToVillage(V3)
 - ├ DropPackage(P3)
 - └ ReturnToWarehouse

HTN Diagram: Multi-Agent Drone Delivery



Highlights of this example

- Shows **hierarchical decomposition** clearly (strategic → operational → primitive).
- Demonstrates **parallel execution** and **coordination** in multi-agent settings.
- Encapsulates **resource constraints**, **battery limits**, and **conflict avoidance**.
- Easy to simulate in a classroom with diagrams showing time steps and paths.

Link for more:

<https://www.geeksforgeeks.org/artificial-intelligence/multiagent-planning-in-ai/>

Utility Theory

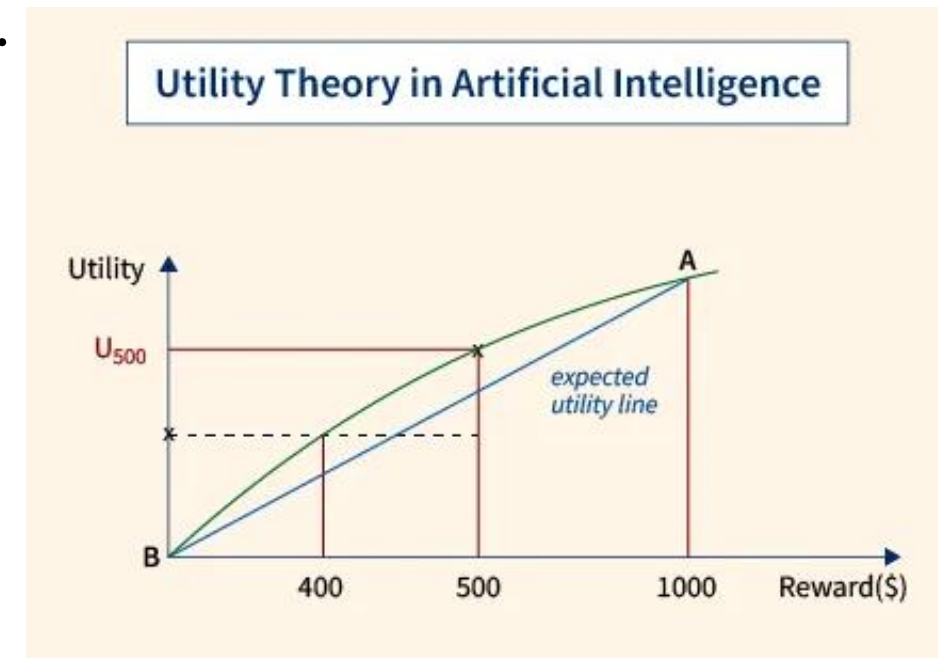
- Utility theory in AI provides a framework for agents to make optimal decisions by maximizing a numerical value representing their preferences, known as utility.
- It helps AI agents choose the best action from a set of options by assigning utility scores to each action and selecting the one with the highest score. This theory is widely applied in various AI domains, including game theory, economics, robotics, and recommendation systems.
- Utility theory is the foundation for **decision-making under uncertainty** in AI. Instead of just *achieving a goal*, we aim to **maximize the overall "goodness"** (utility) of outcomes.

Why Utility Theory?

Planning only works if outcomes are deterministic and equally important but real-world problems often have:

- **Multiple possible outcomes** (due to uncertainty)
- **Different preferences** (some outcomes are better than others)
- **Trade-offs** (you may prefer speed over cost, or vice versa)

Utility theory gives a **quantitative way to compare options** and choose the one with the highest expected satisfaction.



Key Concepts in Utility Theory

1. Utility (U):

- A **numerical score** representing the desirability of an outcome.
- Higher = better.

Example:

- $U(\text{“Safe landing”}) = 100$
- $U(\text{“Landing in storm”}) = 40$
- $U(\text{“Crash”}) = 0$

2. Rational Agent:

- Chooses the action that **maximizes expected utility** given its beliefs.

3. Expected Utility (EU):

If an action has multiple outcomes, each with a probability,

$$EU(a) = \sum_i P(o_i | a) \cdot U(o_i)$$

Example:

Crossing rope bridge in rain:

- Success: $70\% \times \text{Utility}(100) = 70$
- Fall: $30\% \times \text{Utility}(0) = 0$
- **EU = 70 $\rightarrow$ worth crossing? Depends on alternatives.**

Step-by-Step Process of Decision-Making in Utility-Based Agents

1. Perceive the Environment

- The agent gathers information about its current state from the environment.
- This information is used to understand the current situation and to make predictions about the outcomes of potential actions.

2. Generate Possible Actions

- The agent identifies all possible actions it can take from the current state.
- This set of actions is determined by the agent's capabilities and the environment's constraints.

3. Predict Outcomes

- Using the transition model, the agent predicts the resulting states for each possible action.
- The transition model provides the probability or certainty of reaching a specific state from the current state after performing an action.

Contd..

4. Evaluate Utility

- For each predicted state, the agent calculates the utility using the utility function.
- The utility function assigns a numerical value to each state, reflecting its desirability.

5. Select the Optimal Action

- The agent compares the utility values of all predicted states and selects the action that leads to the highest utility.
- This ensures that the agent's choice aligns with its objectives and maximizes overall satisfaction or benefit.

Contd..

6. Act and Observe

- The agent performs the selected action.
- After acting, the agent observes the new state and updates its knowledge about the environment.

7. Learn and Adapt

- Based on the outcomes, the agent may update its utility function or transition model to improve future decision-making.
- Learning allows the agent to adapt to changes in the environment and improve its performance over time.

Example: Working Mechanism of Utility Agents in Intelligent Home Energy System

Consider a utility-based agent designed to manage an intelligent home energy system. The agent's objectives include minimizing energy costs, maximizing comfort, and reducing carbon footprint.

- 1. Perceive the Environment:** The agent collects data from various sensors within the home, such as temperature, humidity, energy consumption, and occupancy status.
- 2. Generate Possible Actions:** The possible actions include adjusting the thermostat, turning appliances on or off, and switching between energy sources (e.g., grid power, solar power).

Contd..

3.Predict Outcomes: Using the transition model, the agent predicts the outcomes for each action.

For example:

- Adjusting the thermostat may lead to a more comfortable temperature but increase energy consumption.
- Turning off unnecessary appliances may reduce energy costs and carbon footprint.
- Switching to solar power may reduce costs and carbon footprint but depend on weather conditions.

Contd..

4. Evaluate Utility: The agent calculates the utility for each predicted outcome. For example:

- Utility may increase with a comfortable temperature but decrease with higher energy costs.
- Utility may increase by reducing energy consumption and carbon footprint.
- Utility may increase with the use of renewable energy sources.

5. Select the Optimal Action: The agent compares the utility values of all predicted outcomes and selects the action that maximizes overall utility. For instance, it might choose to turn off unnecessary appliances and switch to solar power while maintaining a moderate temperature.

Contd..

6. Act and Observe

- The agent implements the selected actions, such as adjusting the thermostat and switching energy sources.
- It then observes the new state of the home, noting changes in temperature, energy consumption, and occupancy.

7. Learn and Adapt

- Based on the observed outcomes, the agent updates its utility function or transition model to improve future decisions.
- For example, it may learn to better predict energy consumption patterns and adjust actions accordingly.

Role of Utility-Based Agents in AI

- 1. Robotics:** Utility-based agents are used to control robots in various tasks, such as investigation, control, and communication between humans and robots. The utility functions help the robots to make decisions that optimize their performance and achieve their goals.
- 2. Finance:** Utility-based agents are used to assist in diagnosis, treatment planning, and personalized medicine. It optimizes their utility, and provides better recommendations and improves patient outcomes.
- 3. Autonomous Vehicles:** Utility-based agents are used in autonomous vehicles for making decisions about navigation, obstacle avoidance, and route planning. These agents ensure safe and efficient travelling by maximizing their utility.
- 4. Game Playing:** Utility-based agents are used in game playing by making strategic decisions, optimizing their performance, and achieving victory. The utility function helps these agents to evaluate different moves and choose the best for the moment.

Utility Function

A utility function maps states or outcomes $\rightarrow$ real numbers that represent preference.

- **Definition:** $U:S \rightarrow \mathbb{R}$

Where S is the set of all possible states.

- **Properties:**

- **Order-preserving:** If $U(s_1) > U(s_2)$, the agent prefers s_1 .
- **Normalization:** Values can be scaled; only the ordering matters.

Example – Autonomous Taxi:

States:

- Passenger delivered on time: $U = 100$
- Passenger slightly late: $U = 70$
- Passenger very late: $U = 20$
- Accident: $U = 0$

Here, the taxi chooses routes and speeds to maximize **expected utility** based on traffic predictions.

Multi-attribute Utility Function

Sometimes, **one number is not enough** — we care about **multiple criteria** at once.

Why?

Real-world decision-making often involves **trade-offs** between attributes:

- Speed vs. Cost
- Risk vs. Reward
- Environmental Impact vs. Profit

Definition:

A **multi-attribute utility function** $U(x_1, x_2, \dots, x_n)$ combines different dimensions into a single preference score.

Types of Multi-attribute Utility Functions

1. Additive Form:

$$U(x_1, x_2) = w_1 \cdot u_1(x_1) + w_2 \cdot u_2(x_2)$$

Attributes are independent.

Example: Drone delivery

- Attribute 1: Speed (weight = 0.6)
- Attribute 2: Safety (weight = 0.4)
- $U = 0.6 \times \text{SpeedScore} + 0.4 \times \text{SafetyScore}$

2. Multiplicative Form:

$$U(x_1, x_2) = k \cdot u_1(x_1) \cdot u_2(x_2)$$

Attributes interact (e.g., if one is 0, total utility drops significantly).

3. Hybrid Form:

Combination of additive and multiplicative terms.

Example: Choosing a Delivery Route for Drones

- **Attributes:**
 - Time (minutes) — prefer lower
 - Battery usage (%) — prefer lower
 - Risk of crash (%) — prefer lower
- **Weights:** Time (0.5), Battery (0.3), Risk (0.2)
- Route A: Time=80, Battery=40, Risk=5
Route B: Time=60, Battery=60, Risk=15
- Use additive formula to compute $U(A)$ and $U(B)$ → choose the higher.

Decision Networks

A **Decision Network** (also called an **Influence Diagram**) is a graphical model that combines:

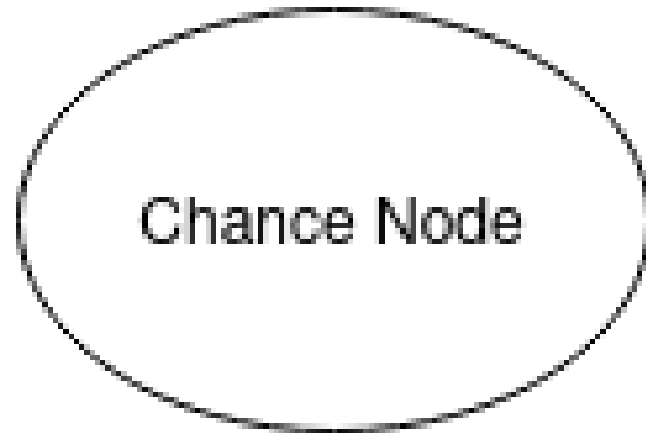
- **Probabilistic Reasoning** (via Bayesian networks)
- **Decision Theory** (actions)
- **Utility Theory** (preferences)
- It's the **formal tool** for making *rational decisions under uncertainty*.

Structure of a Decision Network

A decision network contains three types of nodes:

1. Chance Nodes (Ovals)

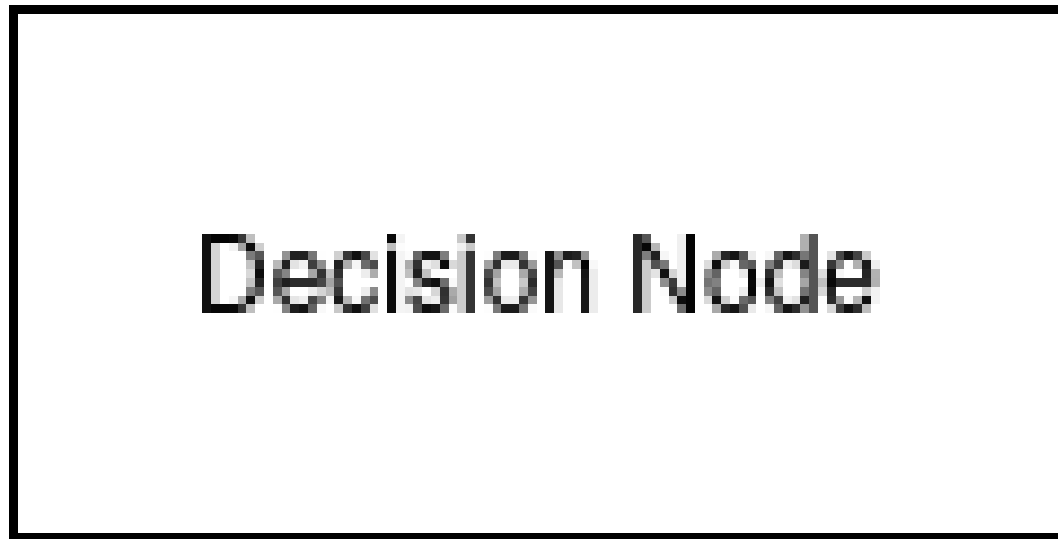
- Represent uncertain variables (random events)
- Labeled with probabilities
- Example: *Weather, Traffic conditions*



Represent random variables and their possible values, capturing the uncertainty in the decision-making process.

2. Decision Nodes (Rectangles)

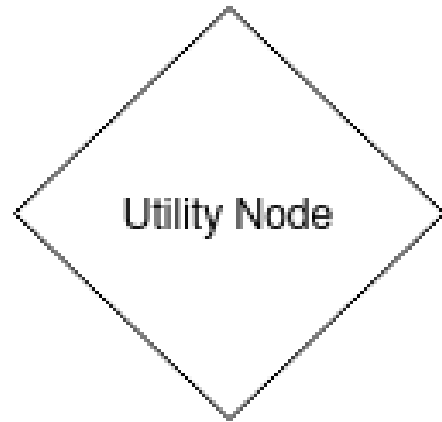
- Represent actions under the agent's control
- Example: *Choose route A or B*



Represent the choices available to the decision-maker.

3. Utility Node (Diamond)

- Represents the utility function
- Outputs the *goodness* of the outcome

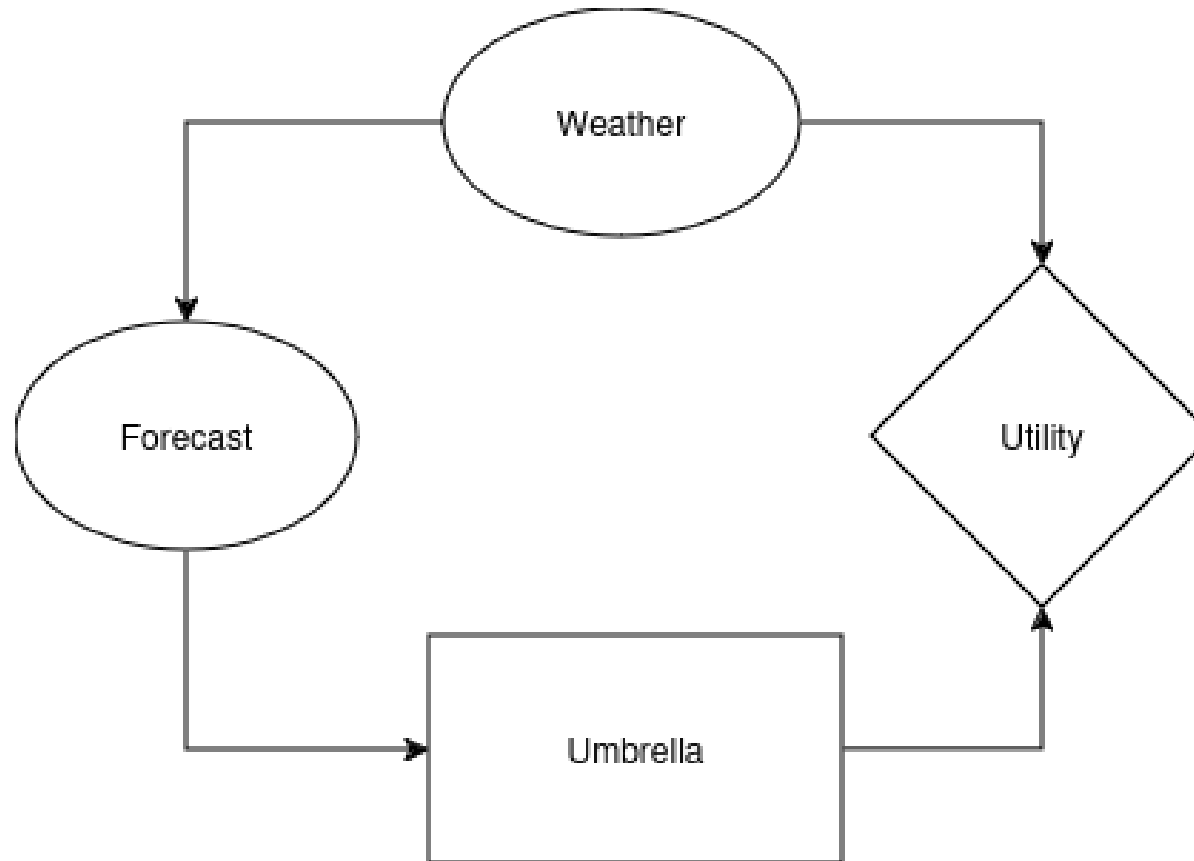


Arcs:

- From chance nodes → show probabilistic dependencies
- From decision nodes → show action influence on states
- To utility node → show which factors affect utility

Example of Representing a Decision Problem

Consider a simple decision network for deciding whether an agent should take an umbrella when going out. The agent's utility depends on the weather and whether he takes an umbrella. The agent can only observe the forecast and does not observe the weather directly. The forecast depends on the weather.



Variables and Domains

- **Weather:** {norain, rain}
- **Forecast:** {sunny, rainy, cloudy}
- **Umbrella:** {take_it, leave_it}

Probabilities

- $P(\text{Weather} = \text{rain}) = 0.3$
- $P(\text{Forecast} \mid \text{Weather})$:

Weather	Forecast	Probability
norain	sunny	0.7
norain	cloudy	0.2
norain	rainy	0.1
rain	sunny	0.15
rain	cloudy	0.25
rain	rainy	0.6

Utility Function

• $u(\text{Weather}, \text{Umbrella})$:

Weather	Umbrella	Utility
norain	take_it	20
norain	leave_it	100
rain	take_it	70
rain	leave_it	0

Example of a Decision Network

Consider a simple medical diagnosis scenario where a doctor needs to decide whether to order a test for a patient based on the likelihood of a disease and the cost of the test. The decision network for this scenario might include:

- **Chance Nodes:** Disease presence (Yes/No), Test result (Positive/Negative)
- **Decision Node:** Order test (Yes/No)
- **Utility Node:** Overall patient health outcome and cost

The doctor can use the decision network to evaluate the expected utility of ordering the test versus not ordering it, taking into account the probabilities of disease presence and test results, and the utility values associated with different outcomes.

Sequential Decision Problems

When decisions are made **over time**, not just once, we have **sequential decision problems**.

Key points:

- The current decision influences **future states**
- Future states influence **future decisions**
- Requires **planning ahead** for maximum *total* utility

Example: Autonomous Taxi Route Planning

1. **State:** Taxi location, passenger location, traffic
2. **Action:** Choose which road to take
3. **Transition Model:** Probability of ending in each possible location after the action
4. **Reward:** Immediate payoff (e.g., -5 for delay, +100 for arrival)

If the taxi only looked at the **immediate** utility, it might take a faster-looking road that gets jammed later. A sequential approach considers **future consequences**.

Algorithms for MDPs

Sequential decision problems with uncertainty are modeled as **Markov Decision Processes (MDPs)**.

MDP Components

1. **States (S)**: All possible configurations of the world
2. **Actions (A)**: All possible moves the agent can make
3. **Transition Model ($P(s'|s,a)$)**: Probability of going to state s' from state s after action a
4. **Reward Function ($R(s)$)**: Immediate reward for being in state s
5. **Policy (π)**: Mapping from states to actions the agent's strategy

Solving MDPs

Two main algorithms:

1. Bellman Optimality Equation for Value Iteration

$$V^*(s) = \max_{a \in A} \sum_{s'} P(s'|s, a) [R(s, a, s') + \gamma V^*(s')]$$

Where:

- $V^*(s) \rightarrow$ **Optimal value function**: the maximum expected total reward from state s , assuming optimal policy.
 - $a \rightarrow$ **Action** that can be taken in state s .
 - $A \rightarrow$ Set of all possible actions.
 - $s' \rightarrow$ **Next state** reached after taking action a in s .
 - $P(s'|s, a) \rightarrow$ **Transition probability**: the probability that next state will be s' given current state s and action a .
 - $R(s, a, s') \rightarrow$ **Reward** received when moving from s to s' via action a .
 - $\gamma \rightarrow$ **Discount factor** ($0 \leq \gamma \leq 1$): determines how much future rewards are valued compared to immediate rewards.
-
- Produces the optimal value for each state.
 - Then derive the best action.

2. Policy Iteration

- Start with an initial policy
- **Policy evaluation:** Compute its value
- **Policy improvement:** Update to a better policy
- Repeat until convergence

Real-life MDP Example: Robot Vacuum

- **States:** Room layout, dirt present or not
- **Actions:** Move north/south/east/west, clean
- **Transition Model:** Sometimes moving fails (probability 0.9 success)
- **Reward:** +10 for cleaning, -1 for moving
- **Policy:** Optimal sequence of moves that maximizes cleaning efficiency over time.

Partially Observable MDPs (POMDPs)

In reality, the agent may not fully observe the state of the world.

POMDP = MDP + Uncertainty in Observation

Components:

- 1. Belief State:** Probability distribution over possible actual states
- 2. Observation Model:** $P(o \mid s, a)$
- 3. Policy:** Maps belief states to actions

Real-life POMDP Example: Search & Rescue Drone

- **State:** Victim location (unknown)
- **Observations:** Thermal camera readings (may be noisy)
- **Actions:** Fly north, south, scan area
- **Reward:** +100 for rescue, -5 for battery usage
- Drone keeps **belief state** (probabilities of where the victim might be) and updates after each observation.

Summary Table:

Concept	Deterministic ?	Fully Observable?	Examples
Planning	Yes	Yes	Classical STRIPS, PDDL
Probabilistic Planning	No	Yes	Weather-affected delivery
MDP	No	Yes	Robot vacuum
POMDP	No	No	Search & rescue drone

Assignment (Self Study): For More About MDP

Possible Questions

1. Explain the representation of actions in AI planning with the help of a real-world domain example, showing clearly how preconditions and effects are defined. Then, represent the same domain using a suitable formal language for planning, including the predicates, initial state, goal state, and actions.
2. Describe how a high-level task from a given domain can be decomposed into sub-tasks until primitive actions are reached, and illustrate this with a hierarchical breakdown.
3. Extend your example to handle uncertainty in action outcomes, specifying the possible states, actions, and transition probabilities, and explain how uncertainty changes the plan compared to a deterministic approach.
4. What is multi-agent planning in AI? Discuss the challenges of coordination and communication between agents. Provide a real-world example involving at least two agents and explain the coordination process step-by-step.
5. Differentiate between single-attribute and multi-attribute utility functions with examples, and show how they can be applied in decision-making under uncertainty.
6. (a) Define a decision network and explain its components with an example.
(b) Write and explain the Bellman Optimality Equation for MDP, defining all terms.